Task 1 — Object-Oriented Inventory System

Brief Documentation • Testing Evidence • Reflection

Implementation Complete	Manual Validation Complete	Automated Tests 25 / 25 Passed	Persistence Verified

1. Objective

Build a practical Python inventory application using object-oriented programming, with classes for products, categories, and inventory operations. The completed system supports add, update, remove, search, validation, JSON persistence, summary reporting, stock-in/stock-out operations, transaction history, audit timestamps, configurable low-stock controls, filtering, and CSV export.

2. Approach and Decisions

- **Object-oriented structure: Category, Product, and StockTransaction classes represent the core entities, while the Inventory class contains business rules and inventory operations.**
- **Separation of responsibilities:** models.py stores data models, inventory.py contains business logic and persistence, and main.py provides the command-line interface.
- **JSON persistence: data/inventory.json stores categories, products, stock transactions, timestamps, and settings. Data is automatically loaded at startup and saved after changes.**
- **Category relationship:** Products reference categories through category IDs to avoid duplicating category information.
- **Validation and error handling:** The application rejects duplicate IDs, unknown categories, invalid prices, negative quantities, empty values, and invalid menu choices without crashing.
- **Testing strategy: Python unittest is used with temporary files so automated tests do not modify the real inventory data. The enhanced final suite contains 25 tests.**
- **Dependencies:** The implementation uses only the Python standard library; no external packages are required.

3. Completed Functionality

Area	Implemented Result
Category management	Add categories with duplicate ID/name validation.
Product management	Add, update, remove, view, and search products with category relationships.
Search	Search by product ID, product name, category ID, or category name.
Validation	Existing-category check, duplicate prevention, positive price, non-negative whole-number quantity.
Persistence	JSON save/load verified across application restarts, including transactions and settings.

Area	Implemented Result
Summary reporting	Total categories, products, units, inventory value, low-stock, out-of-stock, threshold, and transaction counts.
CLI usability	Menu-driven interface with 13 functional options and clear success/error messages.
Stock operations	Stock In and Stock Out with quantity validation and insufficient-stock protection.
Transaction history	Persistent stock movement history with transaction IDs, before/after quantities, and timestamps.
Audit timestamps	created_at / updated_at timestamps on inventory records plus timestamps on stock transactions.
Low-stock controls	Configurable low-stock threshold with low-stock and out-of-stock filtering.
Filtering & export	Filter products by category and export the current inventory to CSV.

4. Testing, Validation, and Evidence

Manual testing was performed through the command-line interface, followed by an automated unittest suite. The following results were observed during the final practical run:

Test Scenario	Observed Result	Status
Add category	Electronics category created successfully.	**PASS**
Add product	TV added under Electronics.	**PASS**
Search	Product 101 found correctly.	**PASS**
Update	TV price changed from 250000 to 230000; quantity from 10 to 8.	**PASS**
Summary	Inventory value recalculated to 1,840,000.00.	**PASS**
Invalid category	Category ID 999 rejected.	**PASS**
Duplicate product ID	Product ID 101 rejected.	**PASS**
Negative price	-1500 rejected.	**PASS**
Negative quantity	-4 rejected.	**PASS**
Remove product	Mouse removed successfully.	**PASS**
Persistence	TV still available after closing and restarting the application.	**PASS**
Stock In	Product 101 quantity increased from 8 to 18; transaction recorded.	**PASS**

Test Scenario	Observed Result	Status
Stock Out	Product 101 quantity decreased from 18 to 14; transaction recorded.	PASS
Insufficient stock	Stock-out request of 500 units rejected without changing inventory.	PASS
Transaction history	STOCK_IN and STOCK_OUT records displayed with IDs, before/after quantities, and timestamps.	PASS
Low-stock threshold	Threshold changed from 5 to 10 and persisted.	PASS
Category filter	Category 101 correctly returned TV under Electronics.	PASS
CSV export	exports/inventory_export.csv generated successfully.	PASS
Audit data	Transaction timestamps and product timestamps created and persisted.	PASS

Automated Test Result

Tests Run	Failures	Errors
25	0	0

Final unittest output: Ran 25 tests in 0.143s - OK

The automated suite covers category and product operations, duplicate prevention, validation, persistence, stock-in/stock-out, insufficient-stock protection, transaction history and persistence, configurable low-stock thresholds, category/stock filtering, CSV export, summary calculations, and audit timestamps.

5. Short Reflection and Next Steps

This task strengthened my understanding of object-oriented design, validation, file persistence, exception handling, audit-friendly stock operations, CSV export, and automated testing in Python. Separating the data models, business logic, user interface, and tests made the project easier to extend, debug, and verify.

If the project is extended further, the next practical improvements would be:

- Migrate JSON storage to SQLite for larger datasets, stronger querying, and safer concurrent updates.
- Add category maintenance, user authentication, and role-based permissions with relationship safeguards.
- Add inventory analytics and visual dashboards for stock movement, value, and low-stock trends.
- Build a GUI or web interface and add automated backup/restore for inventory data.